COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	September 18-23, 2026	**DATE SUBMITTED:**	September 23, 2026
NAME:	Jaime Luis M. Demain	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Activity NO. 9.1 **Tree ADT**

1. PROCEDURE OUTPUT

What procedures you perform?

Code

```cpp
#include <iostream>
#include <vector>
#include <string>

struct TreeNode {
    char val;
    std::vector<TreeNode*> children;

    TreeNode(char data) : val(data) {}
};

// Helper function to add a child
void addChild(TreeNode* parent, TreeNode* child) {
    if (parent != nullptr && child != nullptr) {
        parent->children.push_back(child);
    }
}

int main() {
    // Level 0 (Root)
    TreeNode* nodeA = new TreeNode('A');

    // Level 1
```

```
    TreeNode* nodeB = new TreeNode('B');
    TreeNode* nodeC = new TreeNode('C');
    TreeNode* nodeD = new TreeNode('D');
    TreeNode* nodeE = new TreeNode('E');
    TreeNode* nodeF = new TreeNode('F');
    TreeNode* nodeG = new TreeNode('G');

    addChild(nodeA, nodeB);
    addChild(nodeA, nodeC);
    addChild(nodeA, nodeD);
    addChild(nodeA, nodeE);
    addChild(nodeA, nodeF);
    addChild(nodeA, nodeG);

    // Level 2 & 3
    TreeNode* nodeH = new TreeNode('H');
    addChild(nodeD, nodeH);

    TreeNode* nodeI = new TreeNode('I');
    TreeNode* nodeJ = new TreeNode('J');
    addChild(nodeE, nodeI);
    addChild(nodeE, nodeJ);

    TreeNode* nodeP = new TreeNode('P');
    TreeNode* nodeQ = new TreeNode('Q');
    addChild(nodeJ, nodeP);
    addChild(nodeJ, nodeQ);

    TreeNode* nodeK = new TreeNode('K');
    TreeNode* nodeL = new TreeNode('L');
    TreeNode* nodeM = new TreeNode('M');
    addChild(nodeF, nodeK);
    addChild(nodeF, nodeL);
    addChild(nodeF, nodeM);

    TreeNode* nodeN = new TreeNode('N');
    addChild(nodeG, nodeN);
```

	``
	    std::cout << "General Tree initialized successfully.\n";
	    return 0;
	}
	``

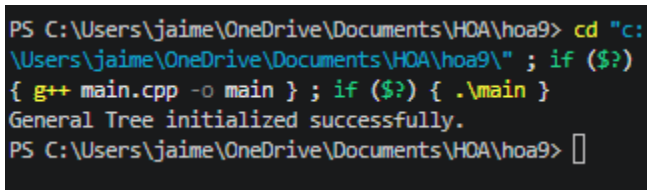
Output	
Observations	Since the branches of general tree nodes vary (Node A has 6 branches, Node E has 2 branches, Node F has 3 branches, while Nodes B & C have 0), using a linked list with dynamic resizing for storing the children (std::vector<TreeNode*>) prevents memory waste which would otherwise occur in case of fixed sized arrays.

Table 9-1. Implementation of General Tree

Node	Height	Depth
A	3	0
B	0	1
C	0	1
D	1	1
E	2	1
F	1	1
G	1	1
H	0	2
I	0	2
J	1	2
K	0	2

L	0	2
M	0	2
N	0	2
P	0	3
Q	0	3

Table 9-2. Node Height and Depth

Traversal Strategy	Path / Order Result
Pre-order	A -> B -> C -> D -> H -> E -> I -> J -> P -> Q -> F -> K -> L -> M -> G -> N
Post-order	B -> C -> H -> D -> I -> P -> Q -> J -> E -> K -> L -> M -> F -> N -> G -> A
In-order	B -> A -> C -> H -> D -> I -> E -> P -> J -> Q -> K -> F -> L -> M -> N -> G

Table 9-3. Manual Traversal Strategies

Code

```
//Traversal function
void preOrder(TreeNode* root) {
    if (!root) return;
    std::cout << root->val << " ";
    for (TreeNode* child : root->children) {
        preOrder(child);
    }
}

void postOrder(TreeNode* root) {
    if (!root) return;
    for (TreeNode* child : root->children) {
        postOrder(child);
    }
```

```
    std::cout << root->val << " ";
}

void inOrder(TreeNode* root) {
    if (!root) return;
    if (root->children.empty()) {
        std::cout << root->val << " ";
        return;
    }
    // Traverse first child
    inOrder(root->children[0]);
    // Visit current node
    std::cout << root->val << " ";
    // Traverse remaining children
    for (size_t i = 1; i < root->children.size(); i++) {
        inOrder(root->children[i]);
    }
}

// Printing traversals to console
    std::cout << "Pre-order:  ";
    preOrder(nodeA);
    std::cout << "\n";

    std::cout << "Post-order: ";
    postOrder(nodeA);
    std::cout << "\n";

    std::cout << "In-order:   ";
    inOrder(nodeA);
    std::cout << "\n";
    return 0;
```

Output

```
Pre-order:  A B C D H E I J P Q F K L M G N
Post-order: B C H D I P Q J E K L M F N G A
In-order:   B A C H D I E P J Q K F L M N G
```

Observations	The output obtained from these two C++ recursive functions is exactly same as the manual derivation through pencil and paper from Table 9-3. Pre-order process the parent before going down to its children, hence it is useful for cloning or structuring purpose. Post-order goes through all of the descendants first and then processes the parent node, hence it is useful in bottom-up tasks such as freeing nodes.

Table 9-4. Tree Traversal Functions Implementation

Code

```
//Search Helper Functions & findData
bool searchPreOrder(TreeNode* root, char key) {
    if (!root) return false;
    if (root->val == key) return true;
    for (TreeNode* child : root->children) {
        if (searchPreOrder(child, key)) return true;
    }
    return false;
}

bool searchPostOrder(TreeNode* root, char key) {
    if (!root) return false;
    for (TreeNode* child : root->children) {
        if (searchPostOrder(child, key)) return true;
    }
    if (root->val == key) return true;
    return false;
}

bool searchInOrder(TreeNode* root, char key) {
    if (!root) return false;
    if (root->children.empty()) return root->val == key;
    if (searchInOrder(root->children[0], key)) return
true;
    if (root->val == key) return true;
    for (size_t i = 1; i < root->children.size(); i++) {
        if (searchInOrder(root->children[i], key)) return
true;
    }
```

```
    return false;
}

void findData(int choice, char key, TreeNode* root) {
    bool found = false;
    switch (choice) {
        case 1: found = searchPreOrder(root, key); break;
        case 2: found = searchPostOrder(root, key); break;
        case 3: found = searchInOrder(root, key); break;
        default: break;
    }
    if (found) {
        std::cout << key << " was found!\n";
    }
}

//findData Demonstrations
    std::cout << "findData Execution\n";
    findData(1, 'J', nodeA); // Choice 1: Pre-order
    findData(2, 'Q', nodeA); // Choice 2: Post-order
    findData(3, 'Z', nodeA); // Choice 3: Non-existent key
(no print)
```

Output

```
findData Execution
J was found!
Q was found!
```

Observations

No matter which strategy is used to implement findData either by using Pre-order, Post-order, or In-order traversal, all the existing nodes within the tree will be examined if the key value is present. The only difference that exists between them is how the nodes will be searched; the search operation on leaf node Q will complete quickly using post-order traversal, while a search on the internal node E will be done quickly using pre-order traversal.

Table 9-5. findData Function Implementation

Code	`//Insert Node 'O' under Node 'G'`
	`std::cout << "Insert 'O' under 'G' & Test\n";`
	`TreeNode* nodeO = new TreeNode('O');`
	`addChild(nodeG, nodeO);`
	
	`// Search for newly inserted node 'O' using In-order`
	`findData(3, 'O', nodeA);`
	
	`// Verify placement in traversal`
	`std::cout << "Updated Pre-order: ";`
	`preOrder(nodeA);`

Output	`Insert 'O' under 'G' & Test`
	`O was found!`
	`Updated Pre-order: A B C D H E I J P Q F K L M G N O`

Observations	After creation of the node 'O' and adding it to the children of nodeG, findData function is able to find the node and prints "O was found!" In the modified traversals, the new node 'O' shows up as a sibling to 'N', and the parent of both nodes is 'G'. The addition of the new node increases the degree of node G to 2 while maintaining the maximum total height of the tree at 3.

Table 9-6. Adding Leaf Node 'O' under Node 'G'

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Step 1: Implement a binary search tree that will take the following values: 2, 3, 9, 18, 0, 1, 4, 5.

```
#include<iostream>

#include"BST.h"
```

```
int main(){

    //NODE

    BinarySearchTree<int>tree(2);



    tree.insert(3);

    tree.insert(9);

    tree.insert(18);

    tree.insert(0);

    tree.insert(1);

    tree.insert(4);

    tree.insert(5);



    std::cout<< "preOrder: ";

    tree.preOrder();

    std::cout<<std::endl;

    std::cout<< "inOrder: ";

    tree.inOrder();

    std::cout<<std::endl;

    std::cout<< "postOrder: ";

    tree.postOrder();

    std::cout<<std::endl;
```

```
    return 0;
}
```

Main.cpp

```
#ifndef BST_H
#define BST_H


template <typename T>
class BinarySearchTree{
    private:
    // create a key
    T key;
    // pointers for left and right
    BinarySearchTree<T>* left;
    BinarySearchTree<T>*right;
    public:
    //constructor
    BinarySearchTree(T value);
    //inster left and right
    void insert(T value);
```

```
    //traversing
    void preOrder();
    void inOrder();
    void postOrder();


    //searching


};


template <typename T>
BinarySearchTree<T>:: BinarySearchTree(T value){
    key = value;
    left = nullptr;
    right = nullptr;
}


template <typename T>
void BinarySearchTree<T>:: insert(T value){


  // whether to insert to the left
```

```
    if (value< key){
        if(left == nullptr){
            left = new BinarySearchTree<T>(value);
        }else{
            left ->insert(value);
        }
    }
// whether to insert to the right
    else if(value>key){
        if(right == nullptr){
            right = new BinarySearchTree<T>(value);
        }else{
            right ->insert(value);
        }
    }
}


template <typename T>
void BinarySearchTree<T>::preOrder(){


    std::cout<< key << " ";
```

```
    if(left != nullptr){

        left ->preOrder();

    }



    if(right != nullptr){

        right->preOrder();

    }

}



template <typename T>

void BinarySearchTree<T>::inOrder(){

    if (left != nullptr){

        left->inOrder();

    }

    std::cout << key << " ";

    if (right != nullptr){

        right->inOrder();

    }

}



template <typename T>
```

```
void BinarySearchTree<T>::postOrder(){

    if (left != nullptr){

        left->postOrder();

    }

    if (right != nullptr){

        right->postOrder();

    }

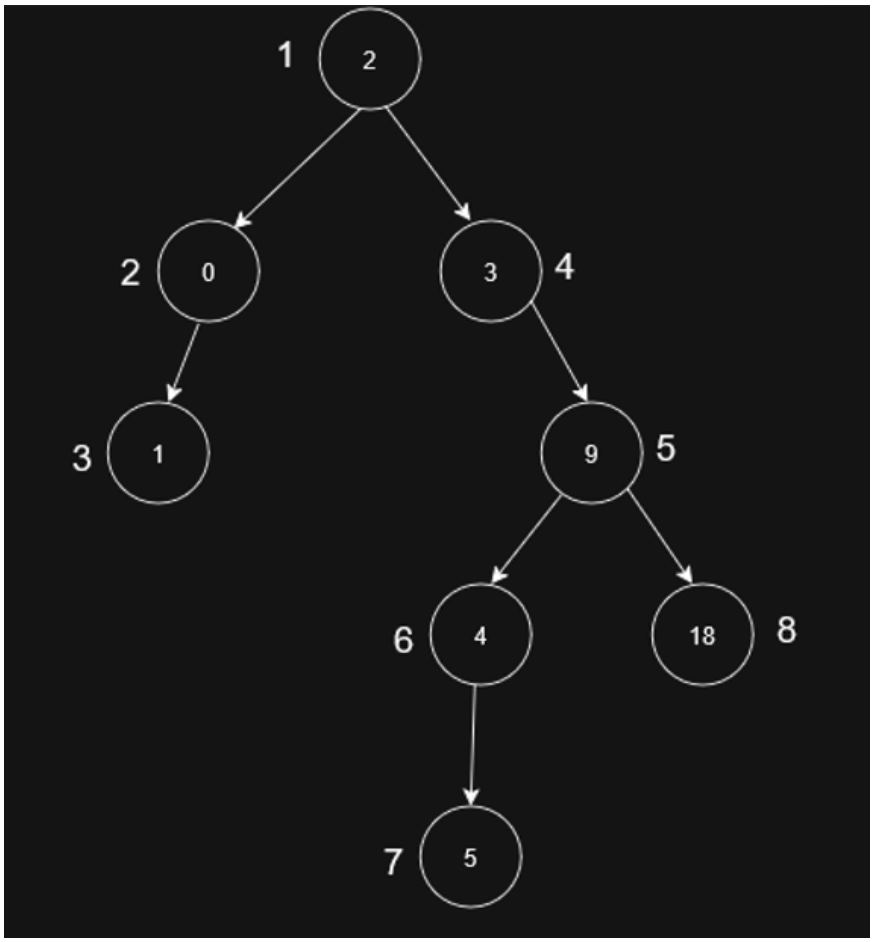
    std::cout << key << " ";

}

#endif
```

BST.h code

Step 2: Create a diagram to show the tree after all values have been inserted. Then, with the use of visual aids (like arrows and numbers) indicate the traversal order for in-order, pre-order and post-order traversal on the diagram.



Step 3: Compare the different traversal methods. In-order traversal was performed with what function?

Answer: In-order traversal was performed using the member function void BinarySearchTree<T>::inOrder() within the BinarySearchTree class template.

```
inOrder: 0 1 2 3 4 5 9 18
```

Output

Observation: This algorithm visits the leftmost child first, prints out the key of the current node, and finally moves to the right subtree (L -> Root -> R). The reason why this works is that the tree has been designed as a Binary Search Tree where all the left descendants are less than the parent and the right descendants are greater.

Pre-order Traversal Section

Created Function:

```
template <typename T>
void BinarySearchTree<T>::preOrder(){
    std::cout << key << " ";
    if (left != nullptr){
        left->preOrder();
    }
    if (right != nullptr){
        right->preOrder();
    }
}
```

```
preOrder: 2 0 1 3 9 4 5 18
```

Output

Observation: The console output follows the same order as that in the Step 2 pre-order traversal. The algorithm works in the order of Root -> Left -> Right. It starts by visiting the root node (2) first, goes to the left subtree (0 and 1), returns to the previous node and finally visits the whole right subtree (3, 9, 4, 5, 18). The order is not numerically ordered since it is in accordance with the hierarchy of the tree.

Post-order Traversal Section

Created Function:

```
template <typename T>
void BinarySearchTree<T>::postOrder(){
    if (left != nullptr){
        left->postOrder();
    }
    if (right != nullptr){
        right->postOrder();
    }
    std::cout << key << " ";
}
```

```
postOrder: 1 0 5 4 18 9 3 2
```

Output

Observation: Output from the console matches the Step 2 post-order diagram. The algorithm proceeds in a Left -> Right -> Root order, whereby the processing of a node cannot take place until all its children have been processed. In the left branch, the leaf node 1 prints out first, followed by 0. In the right branch, the leaf nodes 5 and 18 are processed before their respective parent nodes 4, 9, and 3. Finally, the main root node 2 prints out, hence why post-order is used in bottom-up memory freeing.

Conclusion: The current laboratory exercise enabled hands-on practice on the implementation of non-linear hierarchical data structures in C++ through N-ary trees to Binary Search Trees. In the main exercise, construction of a general tree with varying branch factors was done using the linked nodes, where the computation of height and depth revealed that depth is dependent on the distance from the root, while height is determined by the longest downward path to the leaf node. The process of recursive pre-order, post-order and in-order traversals showed the systematic way of visiting all nodes, which was the driver for the findData function, which helped in searching keys as well as verifying additions like node 'O'. The supplementary exercise involved the construction of a Binary Search Tree with keys {2, 3, 9, 18, 0, 1, 4, 5}, and it showed the effect of ordering properties, such that the keys would be returned in order during an in-order traversal because of strict ordering property, while pre-order and post-order were based on structural dependency. My strengths include recursion on pointers and template classes while weaknesses include dynamic memory deallocation using post-order destructors.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, Quillbot, etc.

Purpose of Use: **For code assistance in some parts of the procedure and forming a supplementary activity to understand its contents and improve grammar in forming an observation and conclusion.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used AI firstly to understand the concepts of the procedure part more and also improve the grammar of the observations of it. Second, to help me understand the supplementary activity. It's because I need to have more clarity on what each item of the supplementary is to answer it precisely. Lastly, I used AI to improve my grammar and tone in forming my conclusion.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.